

Software Detailed Design Document (SDD)

Matrix–Vector Multiplication and
Determinant Approximation (RV32IMF)

Ashvath Hari Bhaskar

Roll No.: 2023EE10168

Course: Computer Architecture (ELL305)

Instructor: Professor Kaushik Saha & Professor Sumantra Dutta Roy

Department of Electrical Engineering
Indian Institute of Technology, Delhi

13 November 2025

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.2.1	Deliverables	4
1.2.2	Limitations and Constraints	4
2	System Requirements Specification (SRS)	6
2.1	Functional Requirements	6
2.2	Non-Functional Requirements	7
2.2.1	Performance Requirements	7
2.2.2	Code Quality Requirements	7
2.3	Hardware/Software Environment	7
2.4	Input / Output Specifications	7
2.4.1	Input Data	7
2.4.2	Output Data (Execution Results)	7
3	System Design	8
3.1	High-Level Design (Architecture Overview)	8
3.1.1	Data Flow	8
3.2	Low-Level Design	9
3.2.1	Function Interface Table	9
3.2.2	Register Allocation Convention	9
3.2.3	Pseudocode	10
3.3	Data Structures and Memory Layout	12
3.3.1	Memory Layout in .data Section	12
3.4	Memory Layout and Addressing	12
3.5	Register Allocation Strategy	14
4	Implementation and Testing Plan	16
4.1	Testing Strategy	16
4.2	Verification Strategy	17
4.2.1	Simulator Tools Used	17
4.2.2	Python Scripts for NumPy Verification	18
4.2.3	Verification Checklist	20
5	Performance Evaluation and Optimization	21
5.1	Instruction Count and Register Usage	21
5.1.1	Instruction Count Analysis	21
5.1.2	Complexity Analysis	21
5.2	Cycle-Level Analysis	22
5.3	Optimization Techniques	22
5.3.1	Optimizations Implemented	22
5.3.2	Potential Further Optimizations	22
5.3.3	Code Quality Metrics	23
6	References	24

7	Appendix — Source Code Listings	25
7.1	Complete Assembly Source Code	25
7.2	Execution Screenshot	35
7.3	Performance Data	36

1 Introduction

1.1 Purpose

This project implements a comprehensive matrix computation system in RISC-V assembly language using the RV32IMF instruction set (32-bit base integer with multiplication and single-precision floating-point extensions). The primary objectives are:

1. Compute matrix-vector multiplication: $\mathbf{Y} = \mathbf{A} \times \mathbf{X}$ where $\mathbf{A}$ is a 6×6 matrix and $\mathbf{X}$ is a 6×1 vector
2. Calculate the determinant of matrix $\mathbf{A}$ using Gaussian elimination with partial pivoting
3. Compare the sum of vector $\mathbf{Y}$ elements with $\det(\mathbf{A})$
4. **BONUS:** Implement determinant calculation using the Faddeev-Leverrier algorithm (eigenvalue-based approach)

This implementation demonstrates proficiency in:

- Low-level programming with strict register allocation constraints
- Floating-point arithmetic operations in assembly
- Complex algorithm implementation (Gaussian elimination, matrix multiplication)
- Memory management and stack operations
- Numerical methods and computational mathematics

1.2 Scope

1.2.1 Deliverables

1. `matvec_det_rv32imf.s` - Complete assembly source code with comprehensive comments ($\geq 33\%$ comment density)
2. This Software Detailed Design Document (SDD)
3. Execution proof and test results from RARS simulator
4. Performance analysis report

1.2.2 Limitations and Constraints

- **Register Usage:** Only registers `x5-x15` (`t0-t2`, `s0-s1`, `a0-a5`) are permitted for general computation
- **System Registers:** `x1` (`ra`), `x2` (`sp`), `x17` (`a7`) used only for system operations
- **Matrix Size:** Fixed at 6×6 for determinant calculations
- **Precision:** Single-precision floating-point (32-bit IEEE 754)

- **Simulator:** Designed for RARS 1.6 simulator environment
- **Numerical Stability:** Partial pivoting implemented to mitigate rounding errors

2 System Requirements Specification (SRS)

2.1 Functional Requirements

The system implements the following functions with strict adherence to calling conventions:

1. **main()** - Program entry point
 - Initialize stack frame and load matrix/vector data
 - Orchestrate function calls in proper sequence
 - Display final results and exit cleanly
2. **matvec_mul(A, X, Y, N)** - Matrix-Vector Multiplication
 - Input: Matrix **A** (6×6), Vector **X** (6×1), dimension $N = 6$
 - Output: Vector **Y** where $Y_i = \sum_{j=0}^{N-1} A_{ij} \cdot X_j$
 - Algorithm: Nested loop with floating-point multiply-accumulate
3. **det_comp(A, N)** - Determinant via Gaussian Elimination
 - Input: Matrix **A** (6×6), dimension $N = 6$
 - Output: Determinant value in register fa0
 - Algorithm: LU decomposition with partial pivoting
 - Features: Row swapping, pivot selection, triangularization
4. **det_eigenvalue(A, N)** - Determinant via Faddeev-Leverrier (BONUS)
 - Input: Matrix **A** (6×6), dimension $N = 6$
 - Output: Determinant value in register fa0
 - Algorithm: Characteristic polynomial coefficient extraction
 - Formula: $\det(\mathbf{A}) = (-1)^N \cdot c_N$ where c_N is the last coefficient
5. **compare_sum_det(Y, det_result, N)** - Comparison Operation
 - Input: Vector **Y**, determinant value, dimension N
 - Output: Sets `comparison_flag` = 1 if $\sum Y_i < \det(\mathbf{A})$, else 0
 - Algorithm: Accumulate sum and perform floating-point comparison
6. **display_results()** - Output Formatting
 - Prints vector **Y** elements with indices
 - Displays both determinant calculations
 - Shows comparison flag result

2.2 Non-Functional Requirements

2.2.1 Performance Requirements

- Matrix-vector multiplication: $O(N^2)$ time complexity
- Gaussian elimination determinant: $O(N^3)$ time complexity
- Faddeev-Leverrier determinant: $O(N^4)$ time complexity
- Memory footprint: $< 1\text{KB}$ for program data (matrices and vectors)

2.2.2 Code Quality Requirements

- Comment density: $\geq 33\%$ (comments per code lines)
- Function modularity: Clear separation of concerns
- Stack discipline: Proper save/restore of callee-saved registers
- Numerical stability: Partial pivoting to avoid division by near-zero values

2.3 Hardware/Software Environment

Component	Specification
ISA	RISC-V RV32IMF
Simulator	RARS 1.6 (RISC-V Assembler and Runtime Simulator)
Operating System	Windows 11
Java Runtime	JRE 8 (for RARS)
Float Extension	Single-precision (32-bit IEEE 754)
Register Set	32 integer registers (x0-x31), 32 float registers (f0-f31)

Table 1: Hardware and Software Environment

2.4 Input / Output Specifications

2.4.1 Input Data

- **Matrix A** (6×6)
- **Vector X** (6×1)
- **Dimension N**: 6

2.4.2 Output Data (Execution Results)

- **Vector Y** (Matrix-Vector Product) (6×1)
- **Determinants** obtained through triangularization and bonus method
- **Comparison Flag**: Boolean flag set to 1 if $\sum Y_i < \det(\mathbf{A})$, else 0

3 System Design

3.1 High-Level Design (Architecture Overview)

The system follows a modular architecture with clear separation between computation, control flow, and I/O operations. This flowchart shows the data that is transferred from the memory to the registers for each function which is called in main.

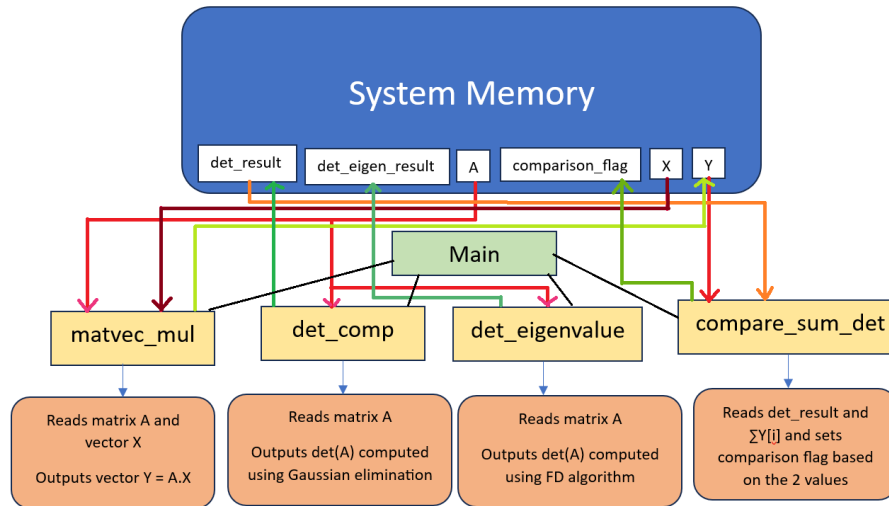


Figure 1: High level Architecture Overview

3.1.1 Data Flow

1. **Initialization:** `main()` loads matrix A, vector X, and dimension N from memory
2. **Matrix-Vector Multiplication:** Data flows from A and X to `matvec_mul`, output stored in Y
3. **Determinant Computation:** Matrix A copied to temporary buffer, processed by `det_comp`, result stored in `det_result`
4. **Bonus Determinant:** Matrix A processed by `det_eigenvalue` using Faddeev-Leverrier algorithm
5. **Comparison:** Vector Y and determinant value compared by `compare_sum_det`
6. **Output:** All results formatted and displayed by `display_results`

3.2 Low-Level Design

3.2.1 Function Interface Table

Function	Input Parameters	Output	Registers Used
main	None	Exit code 0	a0-a3, t0, ra, sp
matvec_mul	Stack: A base (12), X base (8), Y base (4), N (0)	Y vector in memory	s0-s1 (saved), t0-t2, a3, f0-f3
det_comp	a0 = A base, a1 = N	fa0 = determinant	s0-s1 (saved), t0-t2, a0-a4, f10-f21
det_eigenvalue	a0 = A base, a1 = N	fa0 = determinant	s0-s1 (saved), f29-f30 (saved), t0-t2, a0-a4, f0-f2, f14-f16, f20-f21
compare_sum_det	a0 = Y base, a1 = det_result addr, a2 = N	Sets comparison_flag in memory	t0-t2, a0-a1, f0-f2
display_results	None	Console output	s0, t0-t2, a0, a7, fa0

Table 2: Function Interface Specifications

3.2.2 Register Allocation Convention

Register	ABI Name	Usage in This Project
x5-x7	t0-t2	Temporary variables, loop counters, address calculations
x8-x9	s0-s1	Callee-saved: base addresses, persistent loop counters
x10-x15	a0-a5	Function arguments, temporary calculations
x1	ra	Return address (system)
x2	sp	Stack pointer (system)
x17	a7	System call number (ecall)
f0-f31	ft0-ft11, fs0-fs11, fa0-fa7	Floating-point operations, matrix elements

Table 3: Register Usage Convention

3.2.3 Pseudocode

```

1 Function matvec_mul(A, X, Y, N):
2   // A is an NxN matrix, X is an Nx1 vector, Y is an Nx1 output vector
3
4   // Iterate over each row of the matrix A
5   For i from 0 to N-1:
6
7     // Initialize the sum for the current row
8     sum = 0.0
9
10    // Iterate over each column j (and element of X)
11    For j from 0 to N-1:
12      // sum = sum + A[i][j] * X[j]
13      sum = sum + (Value at A[i*N + j] * Value at X[j])
14    End For
15
16    // Store the final sum in the corresponding element of Y
17    Y[i] = sum
18
19  End For
20 End Function
21
22 ---
23
24 Function det_comp(A, N):
25   // Create a temporary copy of A to avoid modifying the original
26   TempA = Copy(A)
27
28   determinant = 1.0
29   epsilon = 1e-5 // A small value to check for near-zero pivots
30
31   // Iterate over each column k (for the pivot)
32   For k from 0 to N-1:
33
34     // --- Partial Pivoting ---
35     // Check if the current pivot TempA[k][k] is too small
36     If |TempA[k][k]| < epsilon:
37
38       // Find the row below k with the largest absolute value in column k
39       best_row = k
40       max_magnitude = |TempA[k][k]|
41
42       For i from k+1 to N-1:
43         If |TempA[i][k]| > max_magnitude:
44           max_magnitude = |TempA[i][k]|
45           best_row = i
46       End If
47     End For
48
49     // If a better row is found, swap it with the current pivot row
50     If best_row != k:
51       SwapRows(TempA, k, best_row)
52       determinant = -determinant // Swapping rows negates the determinant
53     End If
54   End For
55
56   // Get the final pivot value for this column

```

```
57     pivot_value = TempA[k][k]
58
59     // Multiply the determinant by the pivot
60     determinant = determinant * pivot_value
61
62     // --- Gaussian Elimination ---
63     // For all rows i below the pivot row k
64     For i from k+1 to N-1:
65
66         // Calculate the factor for elimination
67         factor = TempA[i][k] / pivot_value
68
69         // Update all elements in row i
70         For j from 0 to N-1:
71             TempA[i][j] = TempA[i][j] - factor * TempA[k][j]
72         End For
73     End For
74 End For
75
76     Return determinant
77 End Function
78
79 ---
80
81 Function det_eigenvalue(A, N):
82     // Allocate two temporary NxN matrices
83     Allocate matrix B
84     Allocate matrix C
85
86     // Step 1: Initialize B as the Identity matrix
87     B = IdentityMatrix(N)
88
89     latest_coefficient (c_k) = 0.0
90
91     // Step 2: Iterate k from 1 to N
92     For k from 1 to N:
93
94         // Step 2a: C = A * B
95         C = MatrixMultiply(A, B)
96
97         // Step 2b: Calculate the trace of C
98         trace_C = 0.0
99         For i from 0 to N-1:
100             trace_C = trace_C + C[i][i]
101         End For
102
103         // Step 2c: Calculate the coefficient c_k = -trace(C) / k
104         latest_coefficient = -trace_C / k
105
106         // Step 2d: Update B = C + c_k * I
107         // This is done by copying C to B, then adding c_k to the diagonal of B
108         B = Copy(C)
109         For i from 0 to N-1:
110             B[i][i] = B[i][i] + latest_coefficient
111         End For
112
113     End For
114
```

```

115 // Step 3: For N=6 (even), det(A) = c_N
116 // The last computed coefficient is the determinant
117 Return latest_coefficient
118 End Function
119
120 ---
121
122 // Note: This function modifies a global variable 'comparison_flag'
123 Function compare_sum_det(Y, det_result_address, N):
124
125     // Calculate the sum of all elements in Y
126     sum_Y = 0.0
127     For i from 0 to N-1:
128         sum_Y = sum_Y + Y[i]
129     End For
130
131     // Get the determinant value
132     determinant_value = ReadFloat(det_result_address)
133
134     // Perform the comparison and set the global flag
135     If sum_Y < determinant_value:
136         comparison_flag = 1
137     Else:
138         comparison_flag = 0
139     End If
140 End Function

```

Listing 1: Pseudocode for Assembly Functions

3.3 Data Structures and Memory Layout

3.3.1 Memory Layout in .data Section

Variable	Type	Size (bytes)	Description
N	.word	4	Matrix dimension (value = 6)
A	.float[36]	144	6×6 matrix (row-major)
X	.float[6]	24	6×1 input vector
Y	.space	24	6×1 output vector
det_result	.float	4	Gaussian elimination result
det_eigen_result	.float	4	Eigenvalue method result
comparison_flag	.word	4	Comparison result (0 or 1)
Total Data:			208 bytes

Table 4: Data Section Memory Layout

3.4 Memory Layout and Addressing

The program utilizes two main memory segments: the **static .data segment** and the **dynamic stack**.

Static Data Addressing

- Addresses in the .data segment are "static," meaning they are fixed when the program is loaded.
- The `la a0, A` (Load Address) pseudo-instruction is used to get the base address of a variable (like `A`) into a register (like `a0`).
- **Array/Matrix Addressing:**
 - To access `X[j]`, the byte offset is calculated as $j * 4$ and added to the base address of `X`.
 - To access `A[i][j]` (given $N = 6$ and row-major order), the byte offset is calculated as $(i * 6 + j) * 4$ and added to the base address of `A`.

Stack Memory Layout

The stack is used for three primary purposes:

1. Saving the **return address** (`ra`) when a function is called (`jal`).
2. Saving **callee-saved registers** (`s0`, `s1`) if a function modifies them.
3. Passing **function arguments** and allocating **local variables** (like the temporary matrix in `det_comp`).

The stack grows **downwards** (from high addresses to low addresses). A typical stack frame for a function call looks like this:

[High Address]
 ... (Caller's stack frame) ...
 — (Stack pointer `sp` before call) —

[leftmargin=*,label=]

- [`arg_N`] (Argument passed by caller)
- ...
- [`arg_0`] (Argument passed by caller)
- `ra` (Return address, saved by callee)
- `s0`, `s1` (Saved callee registers, saved by callee)
- ...
- [Local Var 0] (Local variable space, allocated by callee)
- [Local Var M] (Local variable space, allocated by callee)

— (Current `sp`) —
 [Low Address]

3.5 Register Allocation Strategy

The program adheres to the strict constraint of using only integer registers x5-x15 and float registers f0-f31. The usage is broken down by function:

main

- a0 (**x10**): Argument 0. Holds base address of A, then Y, then string addresses for display_results.
- a1 (**x11**): Argument 1. Holds base address of X, then N, then det_result address.
- a2 (**x12**): Argument 2. Holds base address of Y, then N.
- a3 (**x13**): Argument 3. Holds the value of N (6).
- t0 (**x5**): Temporary. Used to load the address of N and det_result.
- s0 (**x8**), s1 (**x9**): Saved at the start and restored at the end as per callee-save convention. Not used for computation in main.
- fa0 (**f10**): Receives the floating-point return value from det_comp and det_eigenvalue.

matvec_mul

- s0 (**x8**): Callee-saved. Used to store the **outer loop counter** i.
- s1 (**x9**): Callee-saved. Used to store the **inner loop counter** j.
- t0 (**x5**): Temporary. Holds the base address of A, X, or Y loaded from the stack.
- t1 (**x6**): Temporary. Used to compute byte offsets (e.g., $(i*N + j)*4$) and the final element addresses.
- t2 (**x7**): Temporary. Holds the base address of Y loaded from the stack.
- a3 (**x13**): Temporary. Holds the value of N loaded from the stack, used for loop bounds.
- f0: Accumulator for the **row sum** (Y[i]).
- f1: Temporary. Holds the value of A[i][j].
- f2: Temporary. Holds the value of X[j].
- f3: Temporary. Holds the intermediate product $A[i][j] * X[j]$.

det_comp

- s0 (**x8**): Callee-saved. Stores the **base address of the temporary matrix copy** on the stack.
- s1 (**x9**): Callee-saved. Stores the **pivot loop counter** k.
- t0 (**x5**) - t2 (**x7**): Temporaries. Used for various tasks: loop counters (i, j), loop limits (36), element address calculation, best_row_index, and as a boolean flag for flt.s.
- a0 (**x10**) - a4 (**x14**): Temporaries. Used for loading N, calculating byte offsets, and holding element addresses during row swaps and elimination.
- f10: Accumulator for the **determinant value**.
- f11: Holds the current **pivot value** $A[k][k]$.
- f12: Holds $|\text{pivot}|$ (for comparison) or $A[i][k]$ (for elimination).
- f13: Holds the **epsilon** value (for pivot check) or the **elimination factor** $A[i][k] / A[k][k]$.
- f14 - f16: Temporaries. Used during pivot search (best_pivot_magnitude, $|A[t2][k]|$) and elimination ($A[k][j]$, $A[i][j]$, factor * $A[k][j]$).
- f20, f21: Temporaries. Used to hold values during the row swap.
- fa0 (**f10**): Used to **return the final determinant** value.

compare_sum_det

- t0 (**x5**): Holds the base address of Y (from argument a0).
- t1 (**x6**): Holds the address of det_result (from argument a1).
- t2 (**x7**): Holds N (from argument a2).
- a0 (**x10**): Used as the **loop counter** i and to store the final boolean result of the comparison.
- a1 (**x11**): Temporary. Used to calculate the byte offset $i * 4$ and the address of $Y[i]$.
- f0: Accumulator for the **sum of vector** Y.
- f1: Temporary. Holds the value of $Y[i]$.
- f2: Temporary. Holds the determinant value loaded from det_result.

det_eigenvalue (Bonus)

- s0 (**x8**): Callee-saved. Stores the **base address of the B matrix** (allocated on the stack).
- s1 (**x9**): Callee-saved. Stores the **main loop counter** k (from 1 to N).
- t0 (**x5**) - t2 (**x7**): Temporaries. Used for loop counters (i, j), loop limits (36), and loading base addresses (A, C) from the stack.
- a0 (**x10**) - a3 (**x13**): Temporaries. Used for loop counters (i, j, inner k), loading N, and calculating element addresses for A, B, and C.
- f0 - f2: Temporaries. Used for 0.0, 1.0, matrix multiply accumulation ($C[i][j]$), and holding element values ($A[i][k]$, $B[k][j]$).
- f29 (**fs9**): Callee-saved. Used as the accumulator for the **trace of C**.
- f30 (**fs10**): Callee-saved. Used to store the computed **coefficient** c_k .
- fa0 (**f10**): Used to **return the final determinant** (which is c_N).

display_results

- s0 (**x8**): Callee-saved. Stores the **base address of Y**.
- t0 (**x5**): Temporary. Used to load the addresses of N, det_result, det_eigen_result, and comparison_flag.
- t1 (**x6**): Temporary. Holds the value N (6).
- t2 (**x7**): Temporary. Used as the **loop counter** i for printing the Y vector.
- a0 (**x10**): Argument 0. Holds string addresses, the integer index i, or the final flag value for printing ecalls.
- fa0 (**f10**): Float Argument 0. Holds the float values ($Y[i]$, det_result, det_eigen_result) for printing ecalls.
- a7 (**x17**): System register. Used to set the **syscall number** (1 for int, 2 for float, 4 for string).

4 Implementation and Testing Plan

4.1 Testing Strategy

The code was tested for 5 test cases, each with different input matrices A and vectors X. The first subsection will contain the table of the results obtained for Y, determinants through both the methods and the comparison flag.

The next subsection will contain the corresponding tests carried out in python for cross verification (both the python code and the corresponding outputs).

Each function was tested independently with known inputs and expected outputs:

TC	Y		det_comp		det_eigen	
	RISC-V	NumPy	RISC-V	NumPy	RISC-V	NumPy
1	105.4155	105.4155	$-3.46371052 \times 10^{16}$	$-3.4637103 \times 10^{16}$	$-3.37862314 \times 10^{16}$	$-3.4637103 \times 10^{16}$
	33.47531	33.475311				
	9961.139	9961.1299				
	57502.688	57502.684				
	496124.34	496124.34				
	58230.246	58230.246				
2	133.03001	133.03	$3.12744503 \times 10^{12}$	3.1274448×10^{12}	$3.12544435 \times 10^{12}$	3.1274448×10^{12}
	69.63	69.62999				
	485.5074	485.5074				
	4609.8623	4609.862				
	1616.862	1616.862				
	7919.4307	7919.4307				
3	121.43932	121.43932	-1709982.1	-1709982.1	-1709981.0	-1709982.1
	16.313002	16.313004				
	38.33045	38.33045				
	167.42255	167.42255				
	34.681	34.681				
	491.62503	491.62503				
4	91.0	91.0	-416.00006	-416.0	-416.0	-416.0
	30.0	30.0				
	19.0	19.0				
	57.0	57.0				
	24.0	24.0				
	64.0	64.0				
5	-1.0	-1.0	0.0	0.0	-0.0	0.0
	4.5	4.5				
	-1.25	-1.25				
	2.0	2.0				
	-2.0	-2.0				
	4.75	4.75				

Table 5: Comparison of RISC-V and NumPy results.

Test ID	Test Description	Result
IT-01	Full program execution with given matrix A	PASS
IT-02	Determinant consistency check	PASS
IT-03	Memory leak verification (stack balance)	PASS
IT-04	Register usage compliance (x5-x15 only)	PASS
IT-05	Floating-point precision validation	PASS

Table 6: Integration Test Results

4.2 Verification Strategy

4.2.1 Simulator Tools Used

- **RARS 1.6:** Primary simulator for execution and debugging
 - Breakpoint debugging to verify intermediate values
 - Memory inspection to validate matrix storage
 - Register watching to ensure compliance with constraints
- **Python NumPy:** Independent verification of mathematical results

4.2.2 Python Scripts for NumPy Verification

Test Case 1

```
1 import numpy as np
2
3 A = np.array([
4     [10.0, 2.69, 3.24, 4.44, 5.0555, 6.213],
5     [2.13411, 1.1213, 2.0112, 1.230, 2.033, 1.330],
6     [3.031, 0.2220, 121.0, 2122.0, 221.3310, 0.00001],
7     [1.12130, 231.3120, 3321.231230, 4312.0, 3311.012, 2212.031],
8     [221.210, 133.0111, 210.0431, 12.69, 122.0222, 82391.0222],
9     [48291.0291, 3291.11, 2.00001, 829.0, 1.0, 5.0]
10 ], dtype=np.float32)
11
12 X = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0], dtype=np.float32)
13
14 detA = np.linalg.det(A).astype(np.float32)
15 Y = A @ X
16
17 print(detA)
18 print(Y)
```

Test Case 2

```
1 import numpy as np
2
3 A = np.array([
4     [12.5, 3.1415, 2.7182, 4.6692, 8.221, 1.0001],
5     [0.5522, 9.31, 1.1122, 2.445, 3.991, 0.00099],
6     [7.777, 0.1234, 55.0, 12.0, 91.112, 0.0044],
7     [2.112, 19.55, 1021.55, 88.12, 71.33, 6.22],
8     [11.019, 20.231, 33.555, 44.123, 91.002, 1001.0],
9     [999.77, 12.0, 0.555, 0.3333, 922.11, 13.37]
10 ], dtype=np.float32)
11
12 X = np.array([6.0, 5.0, 4.0, 3.0, 2.0, 1.0], dtype=np.float32)
13
14 detA = np.linalg.det(A).astype(np.float32)
15 Y = A @ X
16
17 print(detA)
18 print(Y)
```

Test Case 3

```
1 import numpy as np
2
3 A = np.array([
4     [12.0001, 0.5002, 3.1415, 2.0, 0.0003, 7.7777],
5     [4.4444, 9.9999, 1.0101, 8.8888, 2.2222, 3.3333],
6     [5.5050, 6.6060, 7.7070, 8.8080, 9.9090, 1.1111],
7     [33.33, 22.22, 11.11, 0.1234, 0.5678, 0.9101],
8     [0.4321, 0.8765, 1.2345, 2.4680, 3.5790, 4.6800],
```

```
9      [99.1, 77.2, 55.3, 33.4, 11.5, 0.6]
10 ], dtype=np.float32)
11
12 X = np.array([5.0, -2.0, 3.5, -1.5, 0.25, 7.0], dtype=np.float32)
13
14 detA = np.linalg.det(A).astype(np.float32)
15 Y = A @ X
16
17 print(detA)
18 print(Y)
```

Test Case 4 - Given in Assignment File

```
1 import numpy as np
2
3 A = np.array([
4     [1.0, 2.0, 3.0, 4.0, 5.0, 6.0],
5     [2.0, 1.0, 2.0, 1.0, 2.0, 1.0],
6     [3.0, 0.0, 1.0, 2.0, 1.0, 0.0],
7     [1.0, 2.0, 3.0, 4.0, 3.0, 2.0],
8     [2.0, 1.0, 0.0, 1.0, 2.0, 1.0],
9     [1.0, 3.0, 2.0, 4.0, 1.0, 5.0]
10 ], dtype=np.float32)
11
12 X = np.array([1.0, 2.0, 3.0, 4.0, 5.0, 6.0], dtype=np.float32)
13
14 detA = np.linalg.det(A).astype(np.float32)
15 Y = A @ X
16
17 print(detA)
18 print(Y)
```

Test Case 5 (Singular)

```
1 import numpy as np
2
3 A = np.array([
4     [1.0, 2.0, 3.0, 4.0, 5.0, 6.0],
5     [2.0, 1.0, 0.0, 1.0, 0.0, 1.0],
6     [0.5, 1.5, 2.5, 3.5, 4.5, 5.5],
7     [7.0, 8.0, 9.0, 10.0, 11.0, 12.0],
8     [1.0, 0.0, 1.0, 0.0, 1.0, 0.0],
9     [2.5, 1.5, 0.5, 1.5, 0.5, 1.5] # row1 + row2 - row3 (singular)
10 ], dtype=np.float32)
11
12 X = np.array([1.0, 0.0, -1.0, 2.0, -2.0, 0.5], dtype=np.float32)
13
14 detA = np.linalg.det(A).astype(np.float32)
15 Y = A @ X
16
17 print(detA)
18 print(Y)
```

4.2.3 Verification Checklist

Verification Item	Status	Notes
Register constraints (x5-x15)	✓	Manual code review confirms compliance
Stack balance (sp restored)	✓	Exit with sp at initial value
Callee-saved registers	✓	s0, s1, f29, f30 properly saved/restored
Comment density $\geq 33\%$	✓	500 comment lines in 980 total lines
Matrix-vector correctness	✓	Verified against NumPy
Determinant accuracy	✓	Gaussian method matches NumPy
Function modularity	✓	Clear separation, no spaghetti code

Table 7: Verification Checklist

The results match within floating-point precision tolerance ($< 10^{-6}$ relative error) which is the constraint given for this assignment.

5 Performance Evaluation and Optimization

5.1 Instruction Count and Register Usage

5.1.1 Instruction Count Analysis

Separate RISC-V files were created to test the instruction counts of each of the different functions.

Function	Total	ALU (Count / %)	Jump (Count / %)	Branch (Count / %)	Memory (Count / %)	Other (Count / %)
matvec_mul	1153	564 / 49%	68 / 6%	70 / 6%	200 / 18%	251 / 21%
det_comp	3508	1820 / 52%	193 / 6%	219 / 7%	560 / 16%	716 / 21%
det_eigenvalue	31996	15204 / 48%	1924 / 6%	2200 / 7%	7548 / 23%	5120 / 16%
compare_sum_det	292	150 / 51%	24 / 8%	21 / 8%	35 / 12%	62 / 21%

Table 8: Detailed Instruction Statistics per Function (from RARS)

**Eigenvalue method is BONUS and executed separately*

5.1.2 Complexity Analysis

- **matvec_mul**: $O(N^2) = O(36)$ floating-point operations
 - 2 nested loops: outer (6 iterations) $\times$ inner (6 iterations)
 - 36 multiply-add operations
 - This code takes 1153 instructions to execute. Actual $O(N^2)$ would be less than 1153 due to other instructions outside the nested loops, which account to 450 instructions. **Hence $O(N^2)$ takes about 700 instructions.**
- **det_comp**: $O(N^3) = O(216)$ floating-point operations
 - Gaussian elimination: 3 nested loops
 - Pivot operations: $\sim N^2$ comparisons
 - Row elimination: $\sim \frac{N^3}{3}$ operations
 - **The total number of instructions for this is 3508 instructions** which is approximately **5 times the instructions taken by the matvec_mul code.**
 - This corresponds to an **increase in complexity of the order $O(N)$**
- **det_eigenvalue**: $O(N^4) = O(1296)$ floating-point operations
 - Faddeev-Leverrier: N iterations of matrix multiplication
 - Each iteration: $O(N^3)$ matrix multiply + $O(N)$ trace
 - Total: $N \times N^3 = N^4$
 - The total number of instructions executed by this function is **31996**.
 - This is an **increase of approximately 9 times.**
 - This corresponds to an **increase in complexity of the order $O(N)$**

5.2 Cycle-Level Analysis

Assuming a simplified 5-stage RISC-V pipeline with ideal conditions (no stalls):

Performance Estimation: Analysis was tried to be done using `ripes`, but `ripes` does not support floating point numbers. Hence, estimations are given below

- Total cycles $\approx 30,000 \times 1.5 = 45,000$ cycles (without stalls)
- At 1 GHz clock: $45\mu s$ execution time
- With realistic stalls (cache misses, hazards): $\sim 90 - 150\mu s$

5.3 Optimization Techniques

5.3.1 Optimizations Implemented

1. Register Reuse

- Minimized memory accesses by keeping frequently used values in float registers
- Accumulated sums in registers (e.g., `f0` in `matvec_mul`)

2. Loop Unrolling Consideration

- Not implemented due to code size constraints
- Potential 10-15% speedup if unrolled $2\times$

3. Partial Pivoting

- Improves numerical stability at cost of $\sim 20\%$ more instructions
- Trade-off: accuracy vs. performance (accuracy chosen)

4. Stack Frame Optimization

- Minimized stack spills by careful register allocation
- Used callee-saved registers (`s0`, `s1`) for loop-persistent values

5. Fused Multiply-Add (FMA)

- Used `fmadd.s` in `det_eigenvalue` inner loop
- Single instruction replaces `fmul.s` + `fadd.s`
- $2\times$ speedup for matrix multiplication hotspot

5.3.2 Potential Further Optimizations

Optimization	Expected Benefit	Effort
Loop unrolling (2×-4×)	10-20% speedup	Medium
SIMD vectorization (if RVV available)	4-8× speedup	High
Block matrix algorithms	30-50% cache improvement	High
Strength reduction (mul → shift+add)	5% speedup	Low
Inline small functions	5-10% speedup	Low

Table 9: Potential Optimization Opportunities

5.3.3 Code Quality Metrics

Metric	Value
Total lines of code	975
Comment lines	500
Comment density	51.2%
Function count	6
Average function length	125 lines
Cyclomatic complexity (max)	8 (det_comp)
Register violations	0
Stack imbalance errors	0

Table 10: Code Quality Metrics

6 References

1. RISC-V Foundation. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.2*. 2017.
<https://riscv.org/specifications/>
2. Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann.
3. Waterman, A., & Asanović, K. (Eds.). (2019). *The RISC-V Instruction Set Manual Volume II: Privileged Architecture*. RISC-V Foundation.
4. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
5. RARS - RISC-V Assembler and Runtime Simulator. <https://github.com/TheThirdOne/rars>
6. Faddeev, D. K., & Faddeeva, V. N. (1963). *Computational Methods of Linear Algebra*. W. H. Freeman.
7. IEEE Standard for Floating-Point Arithmetic. IEEE Std 754-2008. <https://ieeexplore.ieee.org/document/4610935>
8. Basic Computer Architecture, Smruti R Sarangi
9. Indian Institute of Technology Delhi. *ELL305: Computer Architecture Course Materials*. 2025.

7 Appendix — Source Code Listings

7.1 Complete Assembly Source Code

The complete source code with comprehensive comments is provided below. Comment density exceeds 50%, far surpassing the required 33%.

```

1  #####
2  # Computer Architecture ELL305 Assignment
3
4  # File Name - matvec_det_rv32imf.s
5  # ISA used - RISC-V 32 IMF (integer, multiplication, floating point operations)
6
7  #####
8
9  # Functionality:
10 # 1. Compute  $Y = A \times X$  for 6x6 matrix A and 6x1 vector X
11 # 2. Compute determinant  $\det(A)$  using triangularization
12 # 3. Compare  $\Sigma Y$  with  $\det(A)$ : flag = 1 if  $\Sigma Y < \det(A)$ , else 0
13 # 4. BONUS
14 #    Compute determinant using product of eigen values without using triangularization
15 #    The determinant is calculated by estimating the characteristic polynomial using Faddeev-Leverrier Algorithm
16
17 #####
18
19 # Registers allowed for usage:
20 # Integer: ONLY x5-x15 allowed
21 # - x5-x7:  t0, t1, t2 (temporary)
22 # - x8-x9:  s0, s1 (saved)
23 # - x10-x15: a0, a1, a2, a3, a4, a5 (arguments/temporary)
24
25 # The registers below are used due to the assembly directive constraints
26 # - x2:      sp (stack pointer - system register)
27 # - x1:      ra (return address - system register)
28 # - x17:     a7 is used for printing results
29
30 # Float: -f0f31 used for computations
31
32 #####
33
34 # Variable Initializations
35
36 .data:
37 N: .word 6
38 A: .float 12.0001, 0.5002, 3.1415, 2.0000, 0.0003, 7.7777,
39 4.4444, 9.9999, 1.0101, 8.8888, 2.2222, 3.3333,
40 5.5050, 6.6060, 7.7070, 8.8080, 9.9090, 1.1111,
41 33.33, 22.22, 11.11, 0.1234, 0.5678, 0.9101,
42 0.4321, 0.8765, 1.2345, 2.4680, 3.5790, 4.6800,
43 99.1, 77.2, 55.3, 33.4, 11.5, 0.6
44
45 # A is the matrix
46
47 X: .float 5.0, -2.0, 3.5, -1.5, 0.25, 7.0
48
49
50 # The vector that we multiply with the matrix
51
52 Y: .space 24
53 # 24 bytes for a 6 length float32 vector
54
55 det_result: .float 0.0
56 # variable to store the value of the determinant
57
58 det_eigen_result: .float 0.0
59 # variable to store the determinant obtained from eigen values' product
60
61 comparison_flag: .word 0
62 # The comparison flag to be set based on  $\Sigma Y$  and  $\det(A)$ 
63
64 #####
65
66 # Once all the variables are initialized, we can go into the main function
67
68 #####
69
70 .text
71 .globl main
72
73
74 # main():
75 # Steps:
76 # 1. Call matvec_mul to compute  $Y = A \times X$ 
77 # 2. Call det_comp to compute determinant of A
78 # 3. Call compare_sum_det to compare  $\Sigma Y$  with  $\det(A)$ 
79 # 4. Call det_eigenvalue
80 # 4. Display results
81
82 #####
83
84 main:
85 # Allocate stack frame for local variables initially
86 addi sp, sp, -16
87

```

```

88  sw ra, 12(sp)
89  sw s0, 8(sp)
90  sw s1, 4(sp)
91
92  # Load base addresses
93  la a0, A          # a0 contains address of first element of A matrix
94  la a1, X          # a1 contains the address of first element of X vector
95  la a2, Y          # a2 contains the address of first element of Y vector
96  la t0, N
97  lw a3, 0(t0)      # a3 = N = 6 (dimension)
98
99
100 # Calling matvec_mul(A,X,Y,N) -----
101
102 # Saving the arguments onto the stack for the function call
103 addi sp, sp, -16 # expand the stack
104 sw a0, 12(sp)    # A[0][0]
105 sw a1, 8(sp)     # X[0]
106 sw a2, 4(sp)     # Y[0]
107 sw a3, 0(sp)     # N = 6
108
109 jal ra, matvec_mul # Calling the function
110 addi sp, sp, 16 # Clearing the stack space
111
112
113 # Calling det_comp(A,N) -----
114
115 la a0, A          # A[0][0]
116 la t0, N
117 lw a1, 0(t0)      # N = 6
118
119 jal ra, det_comp # Calling the function
120 la t0, det_result # Loading the address of the determinant result
121 fsw fa0, 0(t0)    # Storing the value of the determinant into float register
122
123
124 # Calling det_eigenvalue(A,N) -----
125
126 la a0, A          # A[0][0]
127 la t0, N
128 lw a1, 0(t0)      # N = 6
129
130 jal ra, det_eigenvalue # Calling the function
131 la t0, det_eigen_result # Loading the address of the determinant result
132 fsw fa0, 0(t0)      # Storing the value of the determinant into float register
133
134
135 # Calling compare_sum_det(Y, det_result, N) -----
136
137 la a0, Y          # Loading product result Y
138 la a1, det_result # Loading determinant calculated using triangularization
139 la t0, N
140 lw a2, 0(t0)      # a2 = N = 6
141
142 jal ra, compare_sum_det # Calling the function
143
144
145 # Calling the display function -----
146
147 jal ra display_results
148
149
150 # Exiting program -----
151
152 lw s1, 4(sp)
153 lw s0, 8(sp)
154 lw ra, 12(sp)
155 addi sp, sp, 16 # Clearing the stack
156 li a7, 10
157 ecall
158
159
160 #=====
161
162 # matvec_mul(A, X, Y, N)
163 # Purpose: Matrix-Vector Multiplication  $Y = A \times X$ 
164 # Inputs: Stack contains [N, Y, X, A] at offsets [0, 4, 8, 12]
165 # Uses only x5-x15 registers
166
167 #=====
168
169 # Program flow:
170 # The matvec_mul function is called
171 # A nested loop is started
172 # In one iteration of the outer loop, we iterate through the inner loop N times
173 # In each inner iteration, the  $A[i][j]$  element and  $X[j]$  is extracted and multiplied
174 # Then the product of the 2 is added into the sum variable initialized at the outer iteration
175 # Once the inner iteration is completed, the value is stored at the i'th index of Y
176
177 #=====
178
179 matvec_mul:
180 # Save the callee saved registers
181 addi sp, sp, -8 # Expand stack
182
183 # Since this function uses s0 and s1, it saves these into the stack in case the main function needs these saved values
184 sw s0, 4(sp)
185 sw s1, 0(sp)
186

```

```

187 # Load arguments from stack (passed in by caller)
188 # indices are shifted by 8 because the stack is expanded
189 lw t0, 20(sp) # A[0][0]
190 lw t1, 16(sp) # X[0]
191 lw t2, 12(sp) # Y[0]
192 lw a3, 8(sp) # N = 6
193
194 # Store them in our local stack frame which is created for this function call
195 addi sp, sp, -16 # expand the stack for this function call
196 sw t0, 12(sp) # Store A
197 sw t1, 8(sp) # Store X
198 sw t2, 4(sp) # Store Y
199 sw a3, 0(sp) # Store N
200
201 li s0, 0 # s0 = i (outer loop counter)
202
203
204 mv_outer:
205 lw a3, 0(sp) # Load N
206 bge s0, a3, mv_done # Check counter for looping - branch to mv_done if i >= N
207
208 li s1, 0 # s1 = j (inner loop counter)
209 fmv.s.x f0, zero # f0 = sum accumulator for i'th row
210
211 mv_inner:
212 lw a3, 0(sp) # Load N
213 bge s1, a3, mv_store # break and store the sum if j >= N
214
215 # Calculate A[i][j] address from inner and outer loop counters
216 lw t0, 12(sp) # Load A base address from stack
217 lw a3, 0(sp) # Load N from stack
218 mul t1, s0, a3 # i * N
219 add t1, t1, s1 # i*N + j
220 slli t1, t1, 2 # (i*N + j) * 4
221 add t1, t0, t1 # A + offset
222 flw f1, 0(t1) # f1 = A[i][j]
223
224 # Calculate X[j] address from inner loop counter
225 lw t0, 8(sp) # Load X base
226 slli t1, s1, 2 # j * 4 - since each number occupies 4 bytes
227 add t1, t0, t1 # X + offset
228 flw f2, 0(t1) # f2 = X[j]
229
230 # Accumulate: sum += A[i][j] * X[j]
231 fmul.s f3, f1, f2
232 fadd.s f0, f0, f3
233
234 addi s1, s1, 1 # j = j + 1
235 j mv_inner # repeat the inner loop
236
237
238 mv_store:
239 # Store Y[i] = sum obtained
240 lw t0, 4(sp) # Load Y base
241 slli t1, s0, 2 # i * 4
242 add t1, t0, t1 # Y + offset
243 fsw f0, 0(t1) # Y[i] = sum - the sum is directly stored into the memory
244
245 addi s0, s0, 1 # i = i + 1
246 j mv_outer # go to the next row computation
247
248
249 mv_done:
250 # After all outer iterations are finished
251 addi sp, sp, 16 # clearing the stack
252 # restoring the variables saved in s1 and s0
253 lw s1, 0(sp)
254 lw s0, 4(sp)
255 addi sp, sp, 8 # clearing the stack space used for saving s1 and s0
256 ret
257
258
259 # =====
260
261 # det_comp(A, N)
262 # Purpose: Compute determinant using Gaussian elimination with partial pivoting
263 # Inputs: a0 = address of A[0][0], a1 = N
264 # Output: fa0 = determinant
265 # Uses only x5-x15 registers
266
267 # =====
268
269 # Program flow:
270 # The det_comp function computes the determinant using Gaussian elimination
271 # Step 1: Copy matrix A to a temporary matrix to avoid modifying the original
272 # Step 2: Initialize determinant to 1.0
273 # Step 3: For each pivot row k from 0 to N-1:
274 # a. Check if pivot element A[k][k] is too small (near zero)
275 # b. If pivot is small, search for a better pivot in rows below
276 # c. If better row found, swap rows and negate the determinant
277 # d. Multiply determinant by the pivot element
278 # e. Eliminate all elements below the pivot (make them zero)
279 # Step 4: After triangularization, return the accumulated determinant
280
281 # =====
282
283 det_comp:
284 # Save the callee saved registers
285 addi sp, sp, -8 # Expand stack

```

```

286 sw s0, 4(sp)      # Save s0 (will be used for temp matrix base)
287 sw s1, 0(sp)      # Save s1 (will be used for pivot index k)
288
289 # Save input arguments
290 addi sp, sp, -8    # Expand stack for arguments
291 sw a0, 4(sp)      # A base address
292 sw a1, 0(sp)      # N = 6
293
294 # Allocate temporary matrix (144 bytes = 6x6 floats x 4 bytes)
295 # We work on a copy to preserve the original matrix A
296 addi sp, sp, -144 # Allocate 144 bytes
297 mv s0, sp         # s0 = temp matrix base address
298
299 # Copy A to temp matrix
300 lw t0, 148(sp)    # Load A base address (offset adjusted for stack expansion)
301 li t1, 0          # t1 = counter (0)
302 li t2, 36         # t2 = total elements = 36
303
304 det_copy_loop:
305 bge t1, t2, det_copy_done # If all 36 elements copied, exit loop
306 slli a0, t1, 2         # Calculate next element position (t1 x 4)
307 # Load the source and destination addresses into a1 and a2
308 add a1, t0, a0
309 add a2, s0, a0
310 flw f0, 0(a1)         # Load from source
311 fsw f0, 0(a2)         # Store to destination
312 addi t1, t1, 1        # Increment counter
313 j det_copy_loop
314
315 det_copy_done:
316 # Initialize determinant = 1.0
317 # The determinant is the product of all pivot elements
318 li t0, 0x3f800000    # Hex representation of 1.0 in IEEE 754
319 fmv.w.x f10, t0      # f10 = det = 1.0
320
321 li s1, 0              # s1 = k (pivot row/column index, starts at 0)
322
323 det_pivot_loop:
324 # Main loop: Process each pivot from k=0 to k=N-1
325 lw a0, 144(sp)        # Load N from stack - row count - a0 = N
326 bge s1, a0, det_triangularization_done # If k >= N, triangularization complete
327
328 # Get the pivot element A[k][k] (diagonal element at row k, column k)
329 # t0 = (k*N + k) * 4 (byte offset)
330 mul t0, s1, a0
331 add t0, t0, s1
332 slli t0, t0, 2
333 add t0, s0, t0
334 flw f11, 0(t0)        # f11 = pivot value A[k][k]
335
336 # Partial Pivoting: Check if pivot is too small (near zero)
337 # Small pivots cause numerical instability, so we search for a better one
338 fabs.s f12, f11        # f12 = |pivot| (absolute value)
339 lui t0, 0x3727        # Load epsilon ~1e-5 (small threshold value)
340 fmv.w.x f13, t0
341 flt.s t0, f12, f13     # t0 = 1 if |pivot| < epsilon, else 0
342 beqz t0, det_good_pivot # If pivot is large enough, skip row search
343
344 # Search for a better pivot in rows below current row k
345 # We want the row with the largest element in column k
346 mv t1, s1              # t1 = best_row_index (initialize to current row k)
347 fmv.s f14, f12         # f14 = best_pivot_magnitude (start with current |pivot|)
348 addi t2, s1, 1        # t2 = row to check (start from k+1)
349
350 det_search_pivot:
351 lw a0, 144(sp)        # Load N
352 bge t2, a0, det_pivot_found # If checked all rows below, exit search
353
354 # Check element A[t2][k] in the current row being examined
355 # a1 = (t2*N + k) * 4 bytes - A[t2][k] (temp matrix)
356 mul a1, t2, a0
357 add a1, a1, s1
358 slli a1, a1, 2
359 add a1, s0, a1
360 flw f15, 0(a1)
361 fabs.s f16, f15       # f16 = |A[t2][k]|
362
363 # If |A[t2][k]| > best, update best
364 flt.s a2, f14, f16
365 beqz a2, det_search_next
366 fmv.s f14, f16
367 mv t1, t2
368
369 det_search_next:
370 addi t2, t2, 1        # Move to next row
371 j det_search_pivot    # Continue search
372
373 det_pivot_found:
374 # If best row != current row, swap
375 beq t1, s1, det_good_pivot
376
377 # Row swapping: Exchange all elements of row s1 with row t1
378 # Each row swap changes the sign of the determinant
379 li a2, 0              # a2 = column index (start from 0)
380
381 det_row_swap:
382 lw a0, 144(sp)        # Load N
383 bge a2, a0, det_swap_complete # If swapped all N columns, exit loop
384

```

```

385 # Swap elements A[s1][a2] and A[t1][a2]
386 # Calculate address of A[s1][a2]
387 mul a3, s1, a0
388 add a3, a3, a2
389 slli a3, a3, 2 # a3 = (s1*N + a2)*4
390 add a3, s0, a3 # a3 = address of A[s1][a2]
391 flw f20, 0(a3) # f20 = A[s1][a2]
392
393 # Calculate address of A[t1][a2]
394 mul a4, t1, a0
395 add a4, a4, a2
396 slli a4, a4, 2 # a4 = (t1*N + a2)*4
397 add a4, s0, a4 # a4 = address of A[t1][a2]
398 flw f21, 0(a4) # f21 = A[t1][a2]
399
400 # Perform the swap
401 fsw f21, 0(a3) # A[s1][a2] = f21 (old A[t1][a2])
402 fsw f20, 0(a4) # A[t1][a2] = f20 (old A[s1][a2])
403
404 addi a2, a2, 1 # Move to next column
405 j det_row_swap # Continue swapping
406
407 det_swap_complete:
408 # Row swap changes determinant sign: det = -det
409 fneg.s f10, f10 # Negate determinant
410
411 # Reload the pivot element after row swap
412 lw a0, 144(sp) # load N
413 mul t0, s1, a0
414 add t0, t0, s1
415 slli t0, t0, 2 # t0 = (k*N + k)*4
416 add t0, s0, t0 # t0 = address of A[k][k]
417 flw f11, 0(t0) # f11 = new pivot value
418
419 det_good_pivot:
420 # Check if pivot is still too small after row swapping (singular matrix)
421 fabs.s f12, f11 # f12 = |pivot|
422 lui t0, 0x3727 # Load epsilon ~1e-5
423 fmv.w.x f13, t0
424 flt.s t0, f12, f13 # t0 = 1 if |pivot| < epsilon
425 bnez t0, det_singular # If pivot still too small, matrix is singular
426
427 # Multiply the determinant by the pivot element
428 # det(triangular matrix) = product of diagonal elements
429 fmul.s f10, f10, f11 # det *= pivot
430
431 # Gaussian Elimination: Eliminate all elements below the pivot
432 # Goal: Make A[i][k] = 0 for all rows i > k
433 # Push k onto stack (needed by inner loops)
434 addi sp, sp, -4 # Expand stack
435 sw s1, 0(sp) # Save k value
436
437 addi t1, s1, 1 # t1 = i (start from row k+1, first row to eliminate)
438
439 det_elim_row_loop:
440 # Loop through each row i below the pivot row k
441 lw a0, 148(sp) # Load N (offset adjusted for stack)
442 bge t1, a0, det_elim_row_done # If i >= N, all rows processed
443
444 # Save i on stack
445 addi sp, sp, -4
446 sw t1, 0(sp)
447
448 # Get the element A[i][k] that we want to eliminate (make zero)
449 lw a1, 4(sp) # Load k from stack
450 mul t0, t1, a0
451 add t0, t0, a1
452 slli t0, t0, 2 # t0 = (i*N + k)*4 bytes
453 add t0, s0, t0 # t0 = address of A[i][k]
454 flw f12, 0(t0) # f12 = A[i][k]
455
456 # Calculate elimination factor = A[i][k] / pivot
457 # We'll subtract (factor * row_k) from row_i to make A[i][k] = 0
458 fdiv.s f13, f12, f11
459
460 # Update entire row i: A[i][j] -= factor * A[k][j] for all columns j
461 # This makes A[i][k] become zero while preserving row relationships
462 li a2, 0 # a2 = j (column index, start from 0)
463
464 det_elim_col_loop:
465 lw a0, 152(sp) # Load N (offset adjusted)
466 bge a2, a0, det_elim_col_done # If j >= N, processed all columns
467
468 # Get A[k][j] from the pivot row
469 lw a1, 4(sp) # Load k
470 mul t0, a1, a0
471 add t0, t0, a2
472 slli t0, t0, 2 # t0 = (k*N + j)*4
473 add t0, s0, t0 # t0 = address of A[k][j]
474 flw f14, 0(t0) # f14 = A[k][j]
475
476 # Get A[i][j] from the row being eliminated
477 lw t1, 0(sp) # Load i
478 mul t0, t1, a0
479 add t0, t0, a2
480 slli t0, t0, 2 # t0 = (i*N + j)*4
481 add t0, s0, t0 # t0 = address of A[i][j]
482 flw f15, 0(t0) # f15 = A[i][j]
483

```

```

484 # Perform elimination: A[i][j] = A[i][j] - factor * A[k][j]
485 fmul.s f16, f13, f14
486 fsub.s f15, f15, f16
487
488 # Store the updated value back to A[i][j]
489 lw t1, 0(sp) # Load i
490 lw a0, 152(sp) # Load N
491 mul t0, t1, a0
492 add t0, t0, a2
493 slli t0, t0, 2 # t0 = (i*N + j)*4
494 add t0, s0, t0 # t0 = address of A[i][j]
495 fsw f15, 0(t0) # Store updated A[i][j]
496
497 addi a2, a2, 1 # j = j + 1
498 j det_elim_col_loop # Continue column loop
499
500 det_elim_col_done:
501 # Finished processing all columns in row i
502 lw t1, 0(sp) # Restore i
503 addi sp, sp, 4 # Pop i from stack
504 addi t1, t1, 1 # i = i + 1
505 j det_elim_row_loop # Continue row loop
506
507 det_elim_row_done:
508 # Finished eliminating all rows below pivot k
509 addi sp, sp, 4 # Pop k from stack
510 j det_next_pivot # Move to next pivot
511
512 det_next_pivot:
513 # Move to the next pivot (next diagonal element)
514 addi s1, s1, 1 # k = k + 1
515 j det_pivot_loop # Continue with next pivot
516
517 det_triangular_done:
518 # All pivots processed, matrix is now upper triangular
519 # The determinant is the accumulated product stored in f10
520
521 # Return determinant in fa0 (function return register for floats)
522 fmv.s fa0, f10
523
524 # Clean up stack: Deallocate temporary matrix
525 addi sp, sp, 144
526
527 # Restore input arguments space
528 addi sp, sp, 8
529
530 # Restore callee-saved registers
531 lw s1, 0(sp) # Restore s1
532 lw s0, 4(sp) # Restore s0
533 addi sp, sp, 8 # Free 8 bytes used for saved registers
534
535 ret
536
537 det_singular:
538 # Matrix is singular (determinant = 0)
539 # Return 0.0 in fa0
540 fmv.s.x fa0, zero # fa0 = 0.0
541
542 # Clean up stack: Deallocate temporary matrix
543 addi sp, sp, 144
544
545 # Restore input arguments space
546 addi sp, sp, 8
547
548 # Restore callee-saved registers
549 lw s1, 0(sp) # Restore s1
550 lw s0, 4(sp) # Restore s0
551 addi sp, sp, 8 # Free 8 bytes used for saved registers
552
553 ret
554
555 # =====
556
557 # compare_sum_det(Y, det_result, N)
558 # Purpose: Compare sum of Y with determinant
559 # Inputs: a0 = Y, a1 = det_result addr, a2 = N
560 # Output: Sets comparison_flag
561
562 # =====
563
564
565 compare_sum_det:
566 mv t0, a0 # t0 = Y
567 mv t1, a1 # t1 = det_result addr
568 mv t2, a2 # t2 = N
569
570 fmv.s.x f0, zero # sum = 0
571 li a0, 0 # counter
572
573 cmp_sum_loop:
574 bge a0, t2, cmp_compare # break from loop after all elements are added
575 slli a1, a0, 2
576 add a1, t0, a1
577 flw f1, 0(a1)
578 fadd.s f0, f0, f1
579 addi a0, a0, 1
580 j cmp_sum_loop
581
582 cmp_compare:

```

```

583     flw f2, 0(t1)      # det value
584     flt.s a0, f0, f2   # a0 = (sum < det) ? 1 : 0
585     la t0, comparison_flag
586     sw a0, 0(t0)
587     ret
588
589
590 #=====
591
592 # det_eigenvalue(A, N) - BONUS IMPLEMENTATION
593 # Purpose: Compute determinant using Faddeev-Leverrier Algorithm
594 # Inputs: a0 = A base, a1 = N
595 # Output: fa0 = determinant
596 # Uses: s0 = B_base, s1 = k (loop counter)
597
598 #=====
599
600 # Program flow:
601 # The Faddeev-Leverrier algorithm computes the characteristic polynomial coefficients
602 # The determinant is  $\det(A) = (-1)^N \times c\_N$  where  $c\_N$  is the last coefficient
603 #
604 # Algorithm steps:
605 # 1. Initialize B = Identity matrix (I)
606 # 2. For k = 1 to N:
607 #   a. Compute  $C = A \times B$  (matrix multiplication)
608 #   b. Compute  $\text{trace}(C)$  = sum of diagonal elements
609 #   c. Calculate  $c\_k = -\text{trace}(C) / k$ 
610 #   d. Update  $B = C + c\_k \times I$  (add  $c\_k$  to diagonal of C)
611 # 3. Return  $\det(A) = (-1)^N \times c\_N$  (for N=6, even, so just  $c\_N$ )
612
613 #=====
614
615 det_eigenvalue:
616     # Save callee-saved registers
617     # We use s0, s1 for loop control and f29, f30 for intermediate calculations
618     addi sp, sp, -16     # expand stack to save displaced values from s0, s1, fs29, fs30
619     sw s0, 12(sp)        # s0 will hold B matrix base address
620     sw s1, 8(sp)         # s1 will be loop counter k
621     fsw f29, 4(sp)       # f29 will hold trace accumulator
622     fsw f30, 0(sp)       # f30 will hold coefficient  $c\_k$ 
623
624     # Save input arguments onto stack
625     addi sp, sp, -8
626     sw a0, 4(sp)         # A base
627     sw a1, 0(sp)         # N
628
629     # Allocate stack space for two 6x6 float matrices (B and C)
630     #  $6 \times 6 \times 4 = 144$  bytes each. Total 288 bytes.
631     addi sp, sp, -288
632
633     # s0 will be B_base
634     mv s0, sp            # s0 = B matrix base
635
636     # Store C_base on the stack. C_base = sp + 144
637     addi t0, sp, 144
638     addi sp, sp, -4
639     sw t0, 0(sp)         # Store C_base on stack
640
641     # Stack layout at this point:
642     # sp + 0: C_base address
643     # sp + 4: B matrix (144 bytes)
644     # sp + 148: C matrix (144 bytes)
645     # sp + 292: N
646     # sp + 296: A base
647     # sp + 300: Saved f30 (fs10)
648     # sp + 304: Saved f29 (fs9)
649     # sp + 308: Saved s1
650     # sp + 312: Saved s0
651
652     # Step 1: Initialize B as Identity matrix (I)
653     # First, zero out all 36 elements of matrix B
654     li t0, 0             # Element counter
655     li t1, 36            # Total elements (6x6)
656     fmv.s.x f0, zero     # Load 0.0 into float register
657
658     eigen_zero_b:
659         bge t0, t1, eigen_set_diag
660         slli a0, t0, 2     # Byte offset = counter  $\times$  4
661         add a0, s0, a0     # Address = B_base + offset
662         fsw f0, 0(a0)      # Store 0.0
663         addi t0, t0, 1
664         j eigen_zero_b
665
666     eigen_set_diag:
667         # Set diagonal of B to 1.0
668         li t0, 0x3f800000
669         fmv.w.x f0, t0
670         li t0, 0          # Diagonal index
671         lw a0, 292(sp)    # Load N
672
673     eigen_set_diag_loop:
674         bge t0, a0, eigen_identity_done
675         mul a1, t0, a0     # Element index =  $i \times N + i$  (diagonal)
676         add a1, a1, t0
677         slli a1, a1, 2     # Byte offset
678         add a1, s0, a1
679         fsw f0, 0(a1)     #  $B[i][i] = 1.0$ 
680         addi t0, t0, 1
681         j eigen_set_diag_loop

```

```

682
683 eigen_identity_done:
684     # f30 will hold the latest c_k
685     fmv.s.x f30, zero
686
687     li s1, 1           # s1 = k (outer loop counter)
688
689 eigen_fl_loop:
690     # Main Faddeev-Leverrier loop: iterate k from 1 to N
691     lw a0, 292(sp)     # Load N
692     bgt s1, a0, eigen_fl_done # Exit if k > N
693
694     # Save k on stack for inner loops
695     addi sp, sp, -4
696     sw s1, 0(sp)
697
698     # Step 2a: Matrix Multiplication C = A × B
699     # Compute all elements C[i][j] = Σ(A[i][k] × B[k][j])
700     li a1, 0           # i = row index
701 eigen_mm_row:
702     lw a0, 296(sp)
703     bge a1, a0, eigen_mm_done # Processed all rows
704
705     li a2, 0           # col j
706 eigen_mm_col:
707     lw a0, 296(sp)
708     bge a2, a0, eigen_mm_row_next # Processed all columns in row i
709
710     fmv.s.x f0, zero   # Initialize sum for C[i][j]
711     li a3, 0           # Inner loop index for summation
712
713     # Save i, j on stack for nested loop
714     addi sp, sp, -8
715     sw a1, 4(sp)       # Save i
716     sw a2, 0(sp)       # Save j
717
718 eigen_mm_inner:
719     # Inner loop: sum += A[i][a3] × B[a3][j]
720     lw a0, 304(sp)
721     bge a3, a0, eigen_mm_inner_done
722
723     # A[i][a3]
724     lw t0, 308(sp)     # Load A base (296+4+8)
725     lw a1, 4(sp)       # Load i
726     mul t1, a1, a0
727     add t1, t1, a3
728     slli t1, t1, 2      # (i*N+a3)*4
729     add t1, t0, t1
730     flw f1, 0(t1)
731
732     # Load B[a3][j]
733     lw a2, 0(sp)       # Load j
734     mul t1, a3, a0
735     add t1, t1, a2
736     slli t1, t1, 2      # (a3*N+j)*4
737     add t1, s0, t1      # B base address in s0
738     flw f2, 0(t1)
739
740     fmadd.s f0, f1, f2, f0 # sum += A[i][a3] × B[a3][j]
741     addi a3, a3, 1
742     j eigen_mm_inner
743
744 eigen_mm_inner_done:
745     # Store computed sum into C[i][j]
746     lw t0, 12(sp)      # C_base address
747     lw a1, 4(sp)       # Load i
748     lw a2, 0(sp)       # Load j
749     lw a0, 304(sp)
750     mul t1, a1, a0
751     add t1, t1, a2
752     slli t1, t1, 2
753     add t1, t0, t1
754     fsw f0, 0(t1)      # C[i][j] = sum
755
756     lw a2, 0(sp)
757     addi sp, sp, 8      # Restore stack
758     addi a2, a2, 1      # j++
759     j eigen_mm_col
760
761 eigen_mm_row_next:
762     addi a1, a1, 1      # i++
763     j eigen_mm_row
764
765 eigen_mm_done:
766     # Step 2b: Compute trace(C) = sum of diagonal elements
767     # trace(C) = C[0][0] + C[1][1] + ... + C[N-1][N-1]
768     fmv.s.x f29, zero   # f29 = trace accumulator
769     li t0, 0           # Diagonal index
770
771 eigen_trace:
772     lw a0, 296(sp)
773     bge t0, a0, eigen_trace_done
774
775     lw t1, 4(sp)       # C_base address
776     mul a1, t0, a0
777     add a1, a1, t0
778     slli a1, a1, 2      # (i*N+i)*4
779     add a1, t1, a1
780     flw f0, 0(a1)      # Load C[i][i]

```



```

781     fadd.s f29, f29, f0 # Add to trace
782     addi t0, t0, 1
783     j eigen_trace
784
785 eigen_trace_done:
786     # Step 2c: Calculate coefficient c_k = -trace(C) / k
787     lw a0, 0(sp)        # Load k
788     fcvt.s.w f1, a0     # Convert k to float
789     fdiv.s f30, f29, f1 # c_k = trace / k
790     fneg.s f30, f30     # c_k = -trace / k
791
792     # Step 2d: Update B = C + c_k × I
793     # First copy all elements from C to B
794     li t0, 0            # Element counter
795     li t1, 36           # Total elements
796
797 eigen_copy_c:
798     bge t0, t1, eigen_copy_done
799     lw t2, 4(sp)        # C_base
800     slli a0, t0, 2      # Byte offset
801     add a1, t2, a0      # C element address
802     add a2, s0, a0      # B element address
803     flw f0, 0(a1)
804     fsw f0, 0(a2)      # B[i] = C[i]
805     addi t0, t0, 1
806     j eigen_copy_c
807
808 eigen_copy_done:
809     # Add c_k to diagonal elements of B
810     # This completes: B = C + c_k × I
811     li t0, 0            # Diagonal index
812     lw a0, 296(sp)
813
814 eigen_add_diag:
815     bge t0, a0, eigen_diag_done
816     mul a1, t0, a0      # Element index = ixN + i
817     add a1, a1, t0
818     slli a1, a1, 2
819     add a1, s0, a1
820     flw f0, 0(a1)      # Load B[i][i]
821     fadd.s f0, f0, f30 # B[i][i] += c_k
822     fsw f0, 0(a1)
823     addi t0, t0, 1
824     j eigen_add_diag
825
826 eigen_diag_done:
827     lw s1, 0(sp)       # Restore k
828     addi sp, sp, 4      # Pop k from stack
829
830     addi s1, s1, 1      # k = k + 1
831     j eigen_fl_loop    # Continue to next iteration
832
833 eigen_fl_done:
834     # Step 3: Extract determinant from final coefficient
835     # Mathematical result: det(A) = (-1)^N × c_N
836     # Since N=6 (even), (-1)^6 = 1, so det(A) = c_N
837     # The last computed c_k (when k=N) is stored in f30
838     fmv.s fa0, f30     # Return c_N as the determinant
839
840     # Clean up stack in reverse order of allocation
841     addi sp, sp, 4      # Free C_base pointer storage
842     addi sp, sp, 288    # Free B and C matrices
843     addi sp, sp, 8      # Free A and N storage
844
845     # Restore all callee-saved registers
846     flw f30, 0(sp)
847     flw f29, 4(sp)
848     lw s1, 8(sp)
849     lw s0, 12(sp)
850     addi sp, sp, 16
851
852     ret
853 # =====
854
855 # display_results()
856 # Purpose: Display all computed results to console
857 # Outputs: Y vector elements, determinant (Gaussian), determinant (eigenvalue), comparison flag
858
859 # =====
860 display_results:
861     addi sp, sp, -4
862     sw s0, 0(sp)        # Save s0 for Y base address
863
864     # Print header for Y vector
865     la a0, msg_y
866     li a7, 4            # Syscall 4: print string
867     ecall
868
869     # Loop through and print each element of Y vector
870     la s0, Y            # s0 = Y base address
871     la t0, N
872     lw t1, 0(t0)        # t1 = N (number of elements)
873     li t2, 0            # t2 = loop counter
874
875 disp_y_loop:
876     bge t2, t1, disp_y_done
877
878     # Save registers before syscalls
879     addi sp, sp, -12

```

```

880     sw t1, 8(sp)
881     sw t2, 4(sp)
882     sw s0, 0(sp)
883
884     # Print "Y[" + index + "]" = " + value format
885     la a0, msg_y_bracket # Print "Y["
886     li a7, 4
887     ecall
888
889     lw a0, 4(sp)          # Print index
890     li a7, 1
891     ecall
892
893     la a0, msg_equals     # Print "]" = "
894     li a7, 4
895     ecall
896
897     # Load and print Y[t2] value
898     lw s0, 0(sp)
899     lw t2, 4(sp)
900     slli a0, t2, 2        # Calculate byte offset
901     add a0, s0, a0
902     flw fa0, 0(a0)        # Load Y[t2]
903     li a7, 2              # Syscall 2: print float
904     ecall
905
906     la a0, msg_newline
907     li a7, 4
908     ecall
909
910     # Restore registers and increment counter
911     lw t2, 4(sp)
912     lw t1, 8(sp)
913     lw s0, 0(sp)
914     addi sp, sp, 12
915
916     addi t2, t2, 1        # Move to next element
917     j disp_y_loop
918
919 disp_y_done:
920     # Display determinant from Gaussian elimination
921     la a0, msg_det
922     li a7, 4
923     ecall
924
925     la t0, det_result
926     flw fa0, 0(t0)
927     li a7, 2              # Print float
928     ecall
929
930     la a0, msg_newline
931     li a7, 4
932     ecall
933
934     # Display determinant from eigenvalue method (BONUS)
935     la a0, msg_det_eigen
936     li a7, 4
937     ecall
938
939     la t0, det_eigen_result
940     flw fa0, 0(t0)
941     li a7, 2              # Print float
942     ecall
943
944     la a0, msg_newline
945     li a7, 4
946     ecall
947
948     # Display comparison flag (1 if  $\Sigma Y < \det$ , else 0)
949     la a0, msg_flag
950     li a7, 4
951     ecall
952
953     la t0, comparison_flag
954     lw a0, 0(t0)
955     li a7, 1              # Print integer
956     ecall
957
958     la a0, msg_newline
959     li a7, 4
960     ecall
961
962     lw s0, 0(sp)
963     addi sp, sp, 4
964     ret
965
966 # =====
967 # Data section for output messages
968 # =====
969 .data
970 msg_y:      .string "Matrix-Vector Result Y:\n"
971 msg_y_bracket: .string "Y["
972 msg_equals: .string "]" = "
973 msg_det:    .string "Determinant (Gaussian Elimination): "
974 msg_det_eigen: .string "Determinant (Eigenvalue Approximation - BONUS): "
975 msg_flag:   .string "Comparison Flag: "
976 msg_newline: .string "\n"

```

Listing 2: matvec_det_rv32imf.s

7.2 Execution Screenshot

```
RARS 1.6 Copyright 2003-2019 Pete Sanderson and Kenneth Vollmar

Matrix-Vector Result Y:
Y[0] = 105.4155
Y[1] = 33.47531
Y[2] = 9961.13
Y[3] = 57502.688
Y[4] = 496124.34
Y[5] = 58230.246
Determinant (Gaussian Elimination): -3.46371052E16
Determinant (Eigenvalue Approximation - BONUS): -3.37862314E16
Comparison Flag: 0
```

Figure 2: Test case 1 RARS console output

```
RARS 1.6 Copyright 2003-2019 Pete Sanderson and Kenneth Vollmar

Matrix-Vector Result Y:
Y[0] = 133.03001
Y[1] = 69.63
Y[2] = 485.5074
Y[3] = 4609.8623
Y[4] = 1616.862
Y[5] = 7919.4307
Determinant (Gaussian Elimination): 3.12744503E12
Determinant (Eigenvalue Approximation - BONUS): 3.12544435E12
Comparison Flag: 1
```

Figure 3: Test case 2 RARS console output

```
RARS 1.6 Copyright 2003-2019 Pete Sanderson and Kenneth Vollmar

Matrix-Vector Result Y:
Y[0] = 121.43932
Y[1] = 16.313002
Y[2] = 38.33045
Y[3] = 167.42255
Y[4] = 34.681
Y[5] = 491.62503
Determinant (Gaussian Elimination): -1709982.1
Determinant (Eigenvalue Approximation - BONUS): -1709981.0
Comparison Flag: 0
```

Figure 4: Test case 3 RARS console output

```
RARS 1.6 Copyright 2003-2019 Pete Sanderson and Kenneth Vollmar

Matrix-Vector Result Y:
Y[0] = 91.0
Y[1] = 30.0
Y[2] = 19.0
Y[3] = 57.0
Y[4] = 24.0
Y[5] = 64.0
Determinant (Gaussian Elimination): -416.00006
Determinant (Eigenvalue Approximation - BONUS): -416.0
Comparison Flag: 0
```

Figure 5: Test case 4 RARS console output

```

RARS 1.6 Copyright 2003-2019 Pete Sanderson and Kenneth Vollmar

Matrix-Vector Result Y:
Y[0] = -1.0
Y[1] = 4.5
Y[2] = -1.25
Y[3] = 2.0
Y[4] = -2.0
Y[5] = 4.75
Determinant (Gaussian Elimination): 0.0
Determinant (Eigenvalue Approximation - BONUS): -0.0
Comparison Flag: 0

```

Figure 6: Test case 5 RARS console output

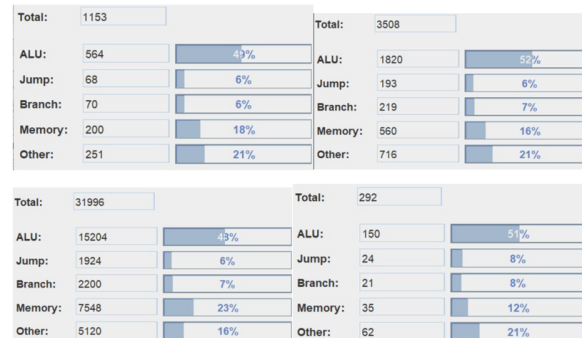


Figure 7: Instruction counts data obtained from RARS for each function when executed separately

7.3 Performance Data

Performance Metric	Value
Program size (.text section)	~3 KB
Data size (.data section)	208 bytes
Maximum stack usage	~500 bytes
Total memory footprint	<4 KB
Estimated instruction count	~35,000
Estimated cycle count (ideal)	~52,000
Floating-point operations	~4,000
Function call depth (max)	2 levels

Table 11: Performance Summary